

Objective of the Exercise

The group should investigate whether there are indications of market manipulation, collusive behavior, spoofing, or information-driven abuse at the account or market level. Students are not expected to prove absolute truth, but rather to derive robust probabilities of suspicion from data patterns.

Data Model and Join Logic

At the core, there are three levels. The market level consists of `markets.csv` and `market_daily_snapshots.csv`. The account level consists of `traders.csv`, `sessions.csv`, `transfers.csv`, and `account_daily_features.csv`. The execution level consists of `orders.csv` and `trades.csv`.

Important keys are `market_id`, `trader_id`, `order_id`, and `trade_id`. `orders.csv` connects accounts with markets. `trades.csv` connects buyers, sellers, and the underlying orders. `sessions.csv` and `transfers.csv` provide infrastructure and money-flow patterns. `account_daily_features.csv` already contains pre-aggregated suspicion indicators on a daily basis and is very suitable for an initial risk score.

Multi-Stage Exercise with 5 Tasks

Task 1: Data Understanding, Data Quality, and Relationship Model

Students load all CSV files into Pandas, check data types, temporal relationships, and primary keys, and create a small data model. They should verify whether `order_id`, `trade_id`, `market_id`, and `trader_id` are unique, where foreign keys are referenced, and which fields must be temporally consistent, such as `created_at <= updated_at` or `session_start <= session_end`.

The expected learning outcome is that the group realizes that a fraud case cannot be derived from a single table. Suspicion only arises through the interaction of market movements, order behavior, counterparties, technical infrastructure, and money flows.

Concrete assignments:

1. Sketch a join plan.
2. Check for missing values and duplicates.
3. Convert the most important time fields into real datetime fields.
4. Mark which fields are likely highly relevant, weakly relevant, or irrelevant noise.

Task 2: Exploratory Analysis of Market and Order Patterns

Students analyze which markets exhibit unusual price and activity patterns. For this, they combine `market_daily_snapshots.csv`, `orders.csv`, and `trades.csv`. The goal is to identify markets with strong price movements despite a low number of distinct buyers or sellers, high cancellation rates, strong concentration on a few accounts, or especially aggressive trades shortly before `close_time`.

The expected learning outcome is an understanding of typical manipulation patterns at the market level. A market is suspicious when price movement and market breadth do not match. A strong price increase with very few participants or with high cancellation intensity is significantly more suspicious than the same price movement in a broad, active market.

Concrete assignments:

- Plot price time series per market.
- Compare daily trading activity and price changes.
- Calculate cancel rates per market.
- Analyze the distribution of counterparties.
- Select and justify a top-10 list of suspicious markets.

Task 3: Account Level and Infrastructure Clusters

Students now switch to the account level. They investigate whether multiple accounts are technically or financially connected. For this, they use `traders.csv`, `sessions.csv`, `transfers.csv`, and `account_daily_features.csv`. Particularly important are shared `device_id`, identical `device_fingerprint_hash`, matching IPs or ASN patterns, high `vpn_share` or `tor_share`, and shared withdrawal addresses.

The expected learning outcome is that fraud is often not a single-account problem, but rather occurs in clusters. Multiple accounts may formally appear different, but use the same technical infrastructure or the same cashout routes.

Concrete assignments:

- Build a network of accounts if they share devices, fingerprints, IPs, or withdrawal addresses.
- Then describe cluster sizes, density, and internal trading shares.
- Finally identify the three most suspicious clusters.

Task 4: Heuristic Fraud Score

Students now develop a transparent risk score. The score should not be a black box, but should consist of clearly interpretable features. Suitable examples include high `cancel_count`, low `fill_ratio_mean`, high `top_counterparty_trade_share`,

high shared_device_session_share, high shared_fp_session_share,
high shared_withdraw_addr_share, high api_order_share, and high self_trade_count.

The expected learning outcome is that good fraud analytics does not begin with complex models. A well-justified, understandable score is pedagogically more valuable for a seminar than immediately applying machine learning. The group should justify weights and then examine which accounts or markets rank highest according to the score.

Concrete assignments:

- Standardize features.
- Document weighting.
- Build an account score and optionally a market score.
- Justify the top cases in written form.

Anyone who wants to may extend the provided `starter_analysis.py` as a starting point.

Task 5: Interpretation, Counter-Hypotheses, and Management Summary

In the final step, students write a short interpretation of the results. They should consciously distinguish between suspicion and proof. For every top case, they must formulate a counter-hypothesis, such as genuine news, low market liquidity, legitimate API usage, or marketing-related traffic spikes. Afterwards, they write a short management summary with the three to five most important suspicious cases and a comprehensible justification.

The expected learning outcome is analytical maturity. A good fraud analysis does not only identify suspicious patterns, but also acknowledges limitations, uncertainties, and alternative explanations.

Didactic Note for Execution

A good sequence for the seminar is:

1. First understand the tables.
2. Then calculate simple descriptive statistics.
3. Afterwards investigate clusters and market patterns.
4. Only at the end build a composite score.

This teaches students that fraud detection consists of hypothesis generation, data understanding, and plausibility assessment.

Clear Explanation of the Individual Data Fields

Below, all fields from the student files are explained individually. Relevance refers to how useful the field is typically for suspicious activity analysis.

markets.csv

`market_id`: Unique primary key of the market. Orders, trades, and daily snapshots are assigned to a market through this field. Relevance: high.

`event_id`: Unique key of the underlying event. Can be used to connect multiple market objects belonging to the same event. Relevance: medium.

`question`: Plain-text question of the market. Shows what exactly is being bet on and helps with domain understanding. Relevance: medium.

`category`: Main category of the market, for example Tech or Politics. More context than a fraud signal. Relevance: low.

`subcategory`: More detailed category within the main category. Relevance: low.

`created_at`: Time when the market was opened or created. Useful for calculating market lifetime. Relevance: medium.

`close_time`: Time when the market closes for new orders. Very important for analyses of activity shortly before market close. Relevance: high.

`resolve_time`: Time of the official market resolution. Helps distinguish pre-resolution and post-resolution phases. Relevance: medium.

`initial_yes_price`: Starting price of the YES side. Serves as the baseline for price movement analysis. Relevance: medium.

`tick_size`: Smallest allowed price increment. Important for determining whether price movements are mechanically plausible. Relevance: medium.

`max_incentive_spread_bps`: Maximum spread in basis points still relevant for incentives. Context for liquidity structure. Relevance: low.

`liquidity_usdc_seeded`: Initially provided liquidity in USDC. Higher initial liquidity generally makes manipulation more difficult. Relevance: high.

`fee_bps`: Trading fee in basis points. Relevant context for trading behavior and market microstructure. Relevance: low.

`market_type`: Market type, typically binary here. Important for correct interpretation of the price space. Relevance: medium.

`outcome_yes_token_id`: Technical token ID for the YES side. Usually not relevant for seminar analyses. Relevance: low.

`outcome_no_token_id`: Technical token ID for the NO side. Usually not relevant for seminar analyses. Relevance: low.

`builder_rewards_enabled`: Indicates whether builder rewards are active. May influence behavior through incentive systems. Relevance: low.

`featured_flag`: Whether the market was prominently featured on the platform. Can artificially increase volume and visibility. Relevance: medium.

`tag_primary`: First tag or keyword of the market. Context only. Relevance: low.

`tag_secondary`: Second tag or keyword of the market. Context only. Relevance: low.

`timezone_hint`: Hint about the used timezone. Relevant for correct time interpretation. Relevance: medium.

`editorial_priority_score`: Internal priority score for editorial promotion. May influence traffic, but is not a direct fraud signal. Relevance: low.

`thumbnail_theme`: Layout or design attribute of the market image. Typically an irrelevant real-world field. Relevance: irrelevant.

`internal_note_stub`: Artificial internal note stub. Unimportant for analysis purposes, but demonstrates real-world dataset clutter. Relevance: irrelevant.

traders.csv

`trader_id`: Unique primary key of the account. Central join key for all account-related tables. Relevance: high.

`wallet_address`: Wallet address of the account. Can be used for cluster and identity analysis if overlaps exist. Relevance: high.

`signup_ts`: Account registration timestamp. Newly created accounts are often more suspicious in abuse scenarios. Relevance: high.

`home_country`: User's home country according to the profile. Only partially trustworthy, but useful as context. Relevance: medium.

`time_zone`: Profile timezone. Can be compared with actual trading times. Relevance: low.

`preferred_language`: Preferred frontend language. Relevance: low.

`kyc_tier`: Identity verification level of the account. Low or missing KYC levels may increase risk. Relevance: high.

`wallet_provider`: Type of wallet provider or custody model. Relevance: medium.

`referral_source`: Account acquisition channel. May become relevant in suspicious cluster formations. Relevance: low.

`vip_tier`: VIP status. Mostly behavior- or fee-related. Relevance: low.

`maker_fee_bps`: Maker fee of the account in basis points. Relevant context for order behavior. Relevance: low.

`taker_fee_bps`: Taker fee of the account in basis points. Relevant context for aggressive trading. Relevance: low.

`builder_code_seen`: Most recently or frequently seen builder or affiliate identifier. Can help identify coordinated clusters. Relevance: medium.

`risk_country_flag`: Binary flag for a country considered risky by the system. Not proof, but a compliance signal. Relevance: medium.

`marketing_opt_in`: Has no meaningful relevance for fraud detection, but is typical for real user data. Relevance: irrelevant.

`ui_theme`: Selected app theme. Pure noise. Relevance: irrelevant.

`favorite_category`: User's favorite category. May indicate usage interests, but usually weak for fraud. Relevance: low.

`avg_session_length_min_prior_30d`: Average session length over the last 30 days. May help distinguish bot-like from normal behavior. Relevance: medium.

`support_tickets_prior_90d`: Number of support cases. Rather weak auxiliary feature. Relevance: low.

`screen_name_style`: Style of the display name. Practically irrelevant. Relevance: irrelevant.

`avatar_color`: Avatar color. Practically irrelevant. Relevance: irrelevant.

`price_alerts_enabled`: Whether price alerts are enabled. May explain activity, but is not a direct fraud indicator. Relevance: low.

`sessions.csv`

`session_id`: Unique key of an individual usage session. Relevance: high.

`trader_id`: Reference to the associated account. Relevance: high.

`session_start`: Session start time. Central for time patterns and session duration. Relevance: high.

`session_end`: Session end time. Together with `session_start`, important for duration and activity windows. Relevance: high.

`device_id`: Device identifier. Multiple accounts on the same `device_id` are suspicious. Relevance: high.

`device_fingerprint_hash`: Hash of a browser or device fingerprint. Even stronger for cluster formation than a simple `device_id`. Relevance: high.

`device_type`: Device type such as mobile or desktop. Mostly context. Relevance: low.

`os_name`: Device operating system. Can be useful for profiling together with other features. Relevance: low.

`browser_family`: Browser family. Interesting only in combination with fingerprints. Relevance: low.

`app_version`: Used app or web version. Can reveal technical clusters. Relevance: low.

`ip_address`: Public IP address. Very important for multi-account and infrastructure analysis. Relevance: high.

`ip_asn`: Network provider or autonomous system number of the IP. Helps identify hosting or datacenter usage. Relevance: medium.

`ip_country`: Country of the IP geolocation. May differ from the profile country. Relevance: medium.

`vpn_detected`: Flag indicating whether VPN usage was detected. Relevance: high.

`tor_exit_detected`: Flag indicating whether a Tor exit node was detected. Strong risk and obfuscation signal. Relevance: high.

`cookie_age_days`: Cookie age in days. Very young cookies combined with high activity may be suspicious. Relevance: medium.

`pageviews`: Number of pages viewed during the session. More behavior-related. Relevance: low.

`avg_scroll_depth_pct`: Average scroll depth. Typical product analytics field, usually weak for fraud. Relevance: irrelevant.

`battery_pct_at_open`: Battery level at session start. Pure noise. Relevance: irrelevant.

`screen_resolution`: Screen resolution. Only potentially useful for fingerprint enrichment. Relevance: low.

`locale_header`: Language or locale header of the client. Relevance: low.

`ab_test_bucket`: A/B test group. Generally irrelevant for fraud. Relevance: irrelevant.

`push_permission`: Push notification status. Pure product field. Relevance: irrelevant.

`date`: Derived calendar-day field of the session. Useful for daily aggregations. Relevance: medium.

orders.csv

`order_id`: Unique primary key of the order. Relevance: high.

`trader_id`: Account that placed the order. Relevance: high.

`market_id`: Market in which the order was placed. Relevance: high.

`event_id`: Event reference of the order. May contain redundant market information. Relevance: low.

`outcome`: Side of the bet, either YES or NO. Relevance: high.

`side`: Trading direction BUY or SELL. Important for microstructure and intent analysis. Relevance: high.

`order_type`: Order type, for example LIMIT. Important for correctly interpreting price logic. Relevance: medium.

`time_in_force`: Validity duration of the order, such as GTC, IOC, or FAK. Particularly relevant for spoofing patterns. Relevance: high.

`post_only`: Whether the order may only enter the book as a maker order. Helps distinguish genuine liquidity provision from aggressive orders. Relevance: medium.

`created_at`: Time the order was created. Central for temporal patterns. Relevance: high.

`updated_at`: Time of the last modification or final status. Important for order lifetime analysis. Relevance: high.

`price`: Order price per share. Core variable for manipulation and price-impact analysis. Relevance: high.

`size_shares`: Original order size in shares. Relevance: high.

`matched_size_shares`: Actually executed portion of the order. Relevance: high.

`remaining_size_shares`: Unexecuted remainder of the order. Relevance: high.

`status`: Final status of the order, such as FILLED, PARTIAL, CANCELED, or EXPIRED. Relevance: high.

`cancel_reason`: Reason for cancellation. Very important for distinguishing manual, systemic, or tactical cancellations. Relevance: high.

`quote_age_ms`: Age of the market quote at order placement in milliseconds. May reveal technical and tactical patterns. Relevance: medium.

`client_latency_ms`: Latency from client to platform. May support API automation or technical cluster detection. Relevance: medium.

`midpoint_before`: Mid-price immediately before order placement. Used to measure how aggressive the order price was. Relevance: high.

`orderbook_imbalance_before`: Order book imbalance before order placement. Useful for understanding whether someone traded against the book. Relevance: high.

`spread_before_bps`: Spread before order placement in basis points. Relevant for liquidity and impact analysis. Relevance: high.

`builder_code`: Builder or integration code through which the order arrived. Very useful for clustering across multiple accounts. Relevance: high.

`ui_entrypoint`: Specific frontend or widget entry point. Helps distinguish traffic sources. Relevance: medium.

`signature_scheme`: Technical signature scheme. Usually only technical context for fraud. Relevance: low.

`scoring_status`: Internal scoring or review status at order intake. Relevance: medium.

`replace_count_15m`: Number of times the order was replaced or modified in the last 15-minute window. Classic spoofing feature. Relevance: high.

`cancel_count_15m`: Number of cancellations in the last 15-minute window. Classic spoofing feature. Relevance: high.

`account_age_days`: Age of the account at order time. Young accounts with aggressive behavior are often suspicious. Relevance: high.

`wallet_balance_before_usdc`: Wallet balance before order placement. Used to validate the plausibility of order sizes. Relevance: medium.

`is_api_order`: Indicates API-based orders. Very relevant for automated manipulation patterns. Relevance: high.

`price_alert_active`: Whether a price alert was active for the user. Rather weak context field. Relevance: low.

`coupon_banner_seen`: Whether an advertisement banner was seen. Practically irrelevant. Relevance: irrelevant.

`date`: Calendar day of the order. Useful for daily aggregation. Relevance: medium.

trades.csv

`trade_id`: Unique primary key of the executed trade. Relevance: high.

`market_id`: Market in which the trade occurred. Relevance: high.

`event_id`: Event reference of the trade. Relevance: low.

`trade_ts`: Exact execution timestamp of the trade. Relevance: high.

`buyer_trader_id`: Account on the buyer side. Relevance: high.

`seller_trader_id`: Account on the seller side. Relevance: high.

`buy_order_id`: Order ID of the buy order. Relevance: high.

`sell_order_id`: Order ID of the sell order. Relevance: high.

`outcome`: Traded outcome, either YES or NO. Relevance: high.

`price`: Trade execution price. Relevance: high.

`size_shares`: Quantity traded. Relevance: high.

`notional_usdc`: Dollar volume of the trade. Price multiplied by quantity. Relevance: high.

`aggressor_side`: Side that actively triggered the trade. Important for directionality and impact. Relevance: high.

`buyer_fee_usdc`: Buyer fee. Relevance: low.

`seller_fee_usdc`: Seller fee. Relevance: low.

`spread_at_trade_bps`: Spread at the moment of the trade. Shows how liquid the market was. Relevance: high.

`midpoint_before`: Mid-price immediately before the trade. Relevance: high.

`midpoint_after`: Mid-price immediately after the trade. Central for measuring price impact. Relevance: high.

`signed_price_change_bps`: Signed price change in basis points. Very important for ramping and information-driven patterns. Relevance: high.

`onchain_settlement_delay_s`: Delay until on-chain settlement in seconds. Relevance: low.

`match_engine_latency_ms`: System matching latency. Can help identify technical clusters. Relevance: low.

`is_self_trade_flag`: Flag for self-trading or very direct self-execution. Highly suspicious. Relevance: high.

`same_builder_code_flag`: Whether both sides came through the same builder context. Helpful for collusion indicators. Relevance: high.

`price_improvement_bps`: Price improvement compared to the visible order book. More microstructure-related than fraud-related. Relevance: medium.

`queue_position_estimate`: Estimated queue position in the order book. Advanced microstructure feature. Relevance: low.

`seq_num`: Technical sequential number. Useful for temporal sorting and consistency checks. Relevance: medium.

`trade_date`: Calendar day of the trade. Relevance: medium.

transfers.csv

`transfer_id`: Unique primary key of the money transfer. Relevance: high.

`trader_id`: Account to which the deposit or withdrawal belongs. Relevance: high.

`ts`: Timestamp of the transfer. Relevance: high.

`direction`: Deposit or withdrawal. Central for funding and cashout patterns. Relevance: high.

`asset`: Transferred asset, usually USDC. Relevance: medium.

`network`: Blockchain or payment network. Relevance: medium.

`amount_usd`: Transfer amount in USD. Very important for funding patterns. Relevance: high.

`bridge_provider`: Bridge or transfer provider. Can reveal technical clusters. Relevance: medium.

`source_or_destination_address`: Source or destination address. Multiple accounts sharing the same destination address are highly suspicious. Relevance: high.

`gas_fee_usd`: Transaction fee. Usually context only for fraud. Relevance: low.

`time_to_credit_s`: Time until funds are credited. More of an operational process field. Relevance: low.

`payment_rail`: Payment method such as chain, card, or bank. Relevance: medium.

`card_bin_country`: Country of the card BIN. Useful for geographic checks in card payments. Relevance: medium.

`chargeback_flag`: Chargeback flag. Classic risk and abuse signal. Relevance: high.

`aml_screen_hit`: Match in AML screenings. Strong compliance signal. Relevance: high.

`travel_rule_required`: Whether regulatory travel-rule obligations apply. More compliance context. Relevance: low.

`memo_field_length`: Length of the memo field. Usually noise in practice. Relevance: irrelevant.

`deposit_campaign_code`: Deposit campaign code. Marketing or bonus context, usually not a direct fraud signal. Relevance: low.

`date`: Calendar day of the transfer. Relevance: medium.

market_daily_snapshots.csv

`market_id`: Market to which the daily snapshot refers. Relevance: high.

trade_date: Calendar day of the snapshot. Relevance: high.

last_price: Last observed price of the day. Central time series for market analysis. Relevance: high.

volume_shares: Traded share volume during the day. Relevance: high.

volume_usdc: Traded dollar volume during the day. Relevance: high.

trade_count: Number of trades during the day. High activity within a narrow counterparty circle can be suspicious. Relevance: high.

avg_spread_bps: Average spread during the day. A narrow or suddenly widening spread may indicate market quality changes. Relevance: high.

unique_buyers: Number of distinct buyers during the day. Relevance: high.

unique_sellers: Number of distinct sellers during the day. Relevance: high.

price_change_bps_sum: Aggregated price change over the day in basis points. Relevance: high.

order_imbalance_proxy: Approximation of demand or supply imbalance. Relevance: high.

social_mentions_1h: External social media mentions. Can explain genuine information events. Relevance: medium.

news_sentiment_score: Sentiment score of external news. Can justify price movements without fraud. Relevance: medium.

bot_comment_ratio: Share of bot-like comments. Potentially useful for hype or manipulation environments. Relevance: medium.

weather_temp_c: Weather temperature. Intentionally irrelevant field. Relevance: irrelevant.

promo_banner_impressions: Number of advertisement impressions. May explain traffic, but is usually secondary for fraud. Relevance: low.

emoji_reaction_count: Number of product reactions. Typical background noise. Relevance: irrelevant.

account_daily_features.csv

trader_id: Account for which the daily features were aggregated. Relevance: high.

date: Calendar day of the aggregation. Relevance: high.

orders_placed: Number of orders placed during the day. Relevance: high.

cancel_count: Number of canceled orders during the day. Core signal for spoofing. Relevance: high.

expired_count: Number of expired orders. May indicate passive quoting behavior. Relevance: medium.

mean_order_size: Average order size during the day. Relevance: high.

max_order_size: Largest individual order during the day. Relevance: high.

mean_replace_count_15m: Average number of order modifications within a 15-minute window. Relevance: high.

mean_cancel_count_15m: Average number of cancellations within a 15-minute window. Relevance: high.

api_order_share: Share of API-based orders in daily activity. Relevance: high.

fill_ratio_mean: Average execution ratio of orders. Very low values combined with high activity are suspicious. Relevance: high.

buy_trades: Number of buys during the day. Relevance: medium.

buy_volume_usdc: Buy volume in USDC during the day. Relevance: medium.

sell_trades: Number of sells during the day. Relevance: medium.

sell_volume_usdc: Sell volume in USDC during the day. Relevance: medium.

sessions_count: Number of account sessions during the day. Relevance: medium.

distinct_devices: Number of distinct devices during the day. Many devices may be suspicious. Relevance: high.

distinct_ips: Number of distinct IP addresses during the day. Relevance: high.

vpn_share: Share of sessions using VPN during the day. Relevance: high.

tor_share: Share of sessions using Tor during the day. Relevance: high.

same_fp_repeat: Repetition measure of an identical fingerprint. May indicate highly automated usage. Relevance: high.

deposits_usd: Total deposits during the day. Relevance: high.

`withdrawals_usd`: Total withdrawals during the day. Relevance: high.

`top_counterparty_trade_share`: Share of trading volume with the main counterparty. Very high values indicate collusion. Relevance: high.

`self_trade_count`: Number of flagged self-trading events during the day. Relevance: high.

`shared_device_session_share`: Share of sessions on devices also used by other accounts. Relevance: high.

`shared_fp_session_share`: Share of sessions with fingerprints also used by other accounts. Relevance: high.

`shared_withdraw_addr_share`: Share of withdrawals to addresses shared with other accounts. Relevance: high.

Typical Patterns Students Should Search For

A collusive pattern exists when a few accounts repeatedly trade with each other, use the same infrastructure, and simultaneously execute an unusually high share of their volume with only one counterparty.

A spoofing pattern exists when an account places many large orders, allows only minimal execution, and quickly cancels them again.

An information-driven pattern exists when individual accounts repeatedly position themselves correctly shortly before market close or before major price movements, while only a few counterparties are involved.

Small Python Hints for Getting Started

Useful initial metrics are:

- `fill_ratio = matched_size_shares / size_shares`
- `cancel_to_order_ratio = cancel_count / orders_placed`
- `counterparty_concentration = top_counterparty_trade_share`
- `shared_infra_score = shared_device_session_share + shared_fp_session_share + shared_withdraw_addr_share`
- `near_close_minutes = (close_time - trade_ts).dt.total_seconds() / 60`

Expected Submission Format for Students

Recommended deliverables are:

- A short notebook or Python script
- Two to four meaningful visualizations
- A table of the top suspicious cases at account and market level
- A one-page summary of findings including justification and counter-hypotheses